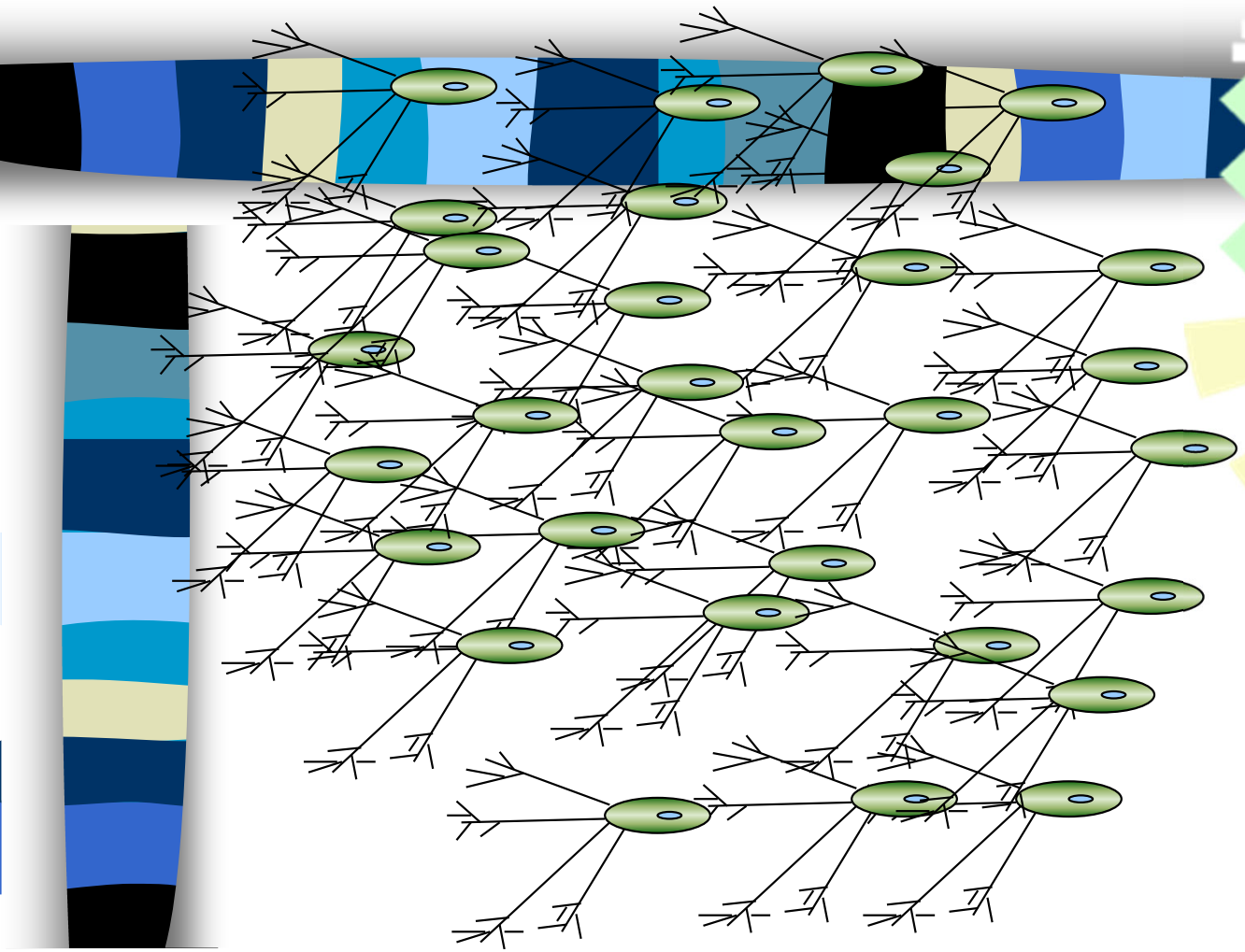
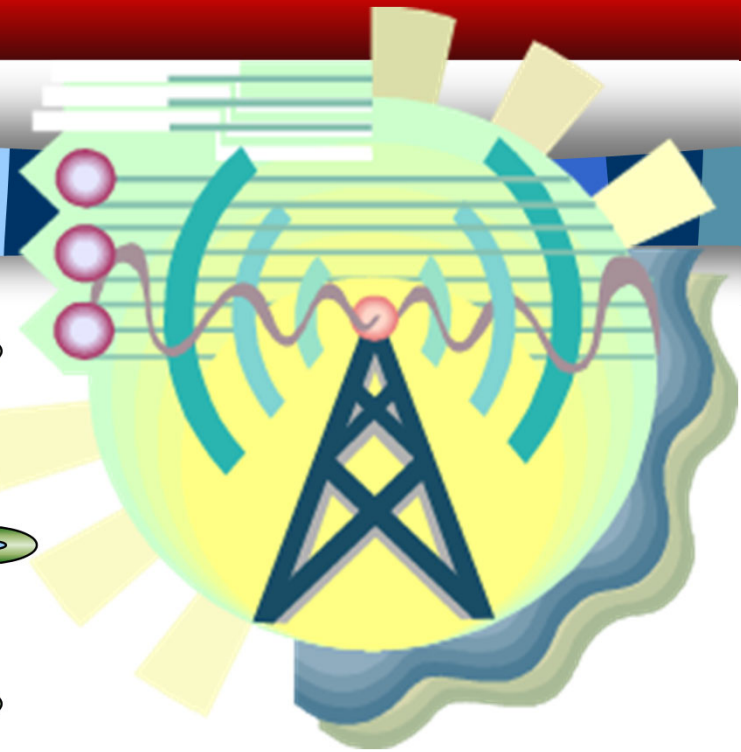
Perceptrons





n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Multi-layer Perceptron

Overview

In order to understand the modern deep learning systems of today, we need to take a step back and learn Perceptrons first.

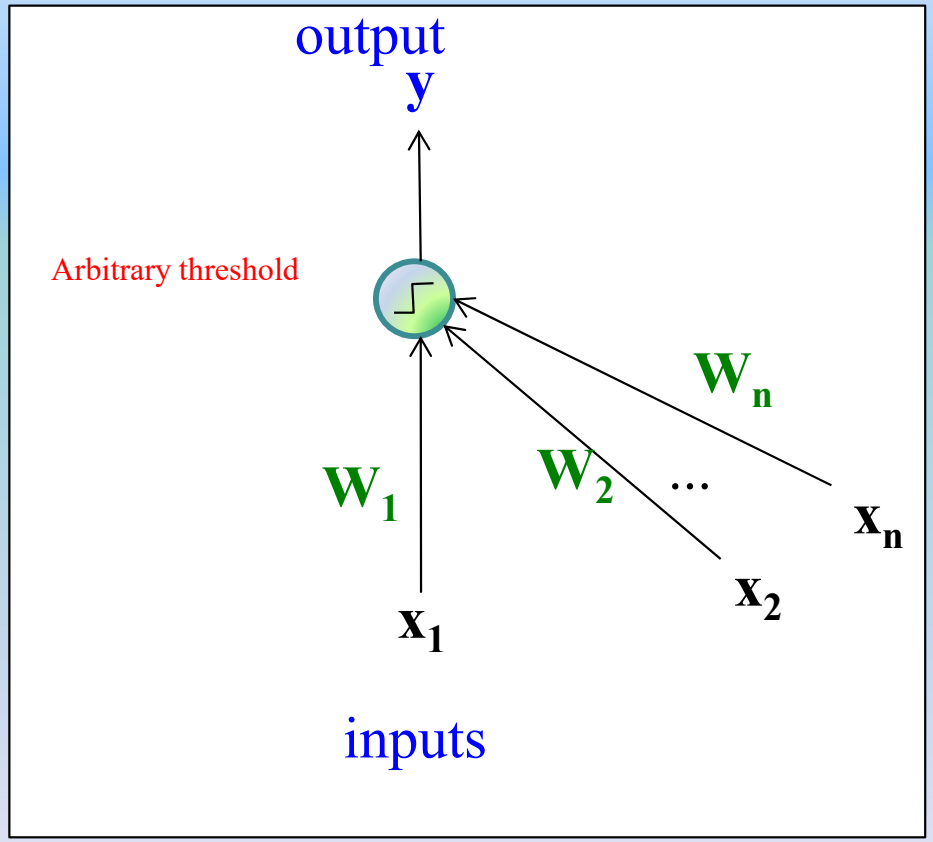
What is the role of a perceptron unit?

What is the advantage of organising them into layers?

What can we achieve if we construct a deep network of perceptron units?

How to train a network of perceptrons?

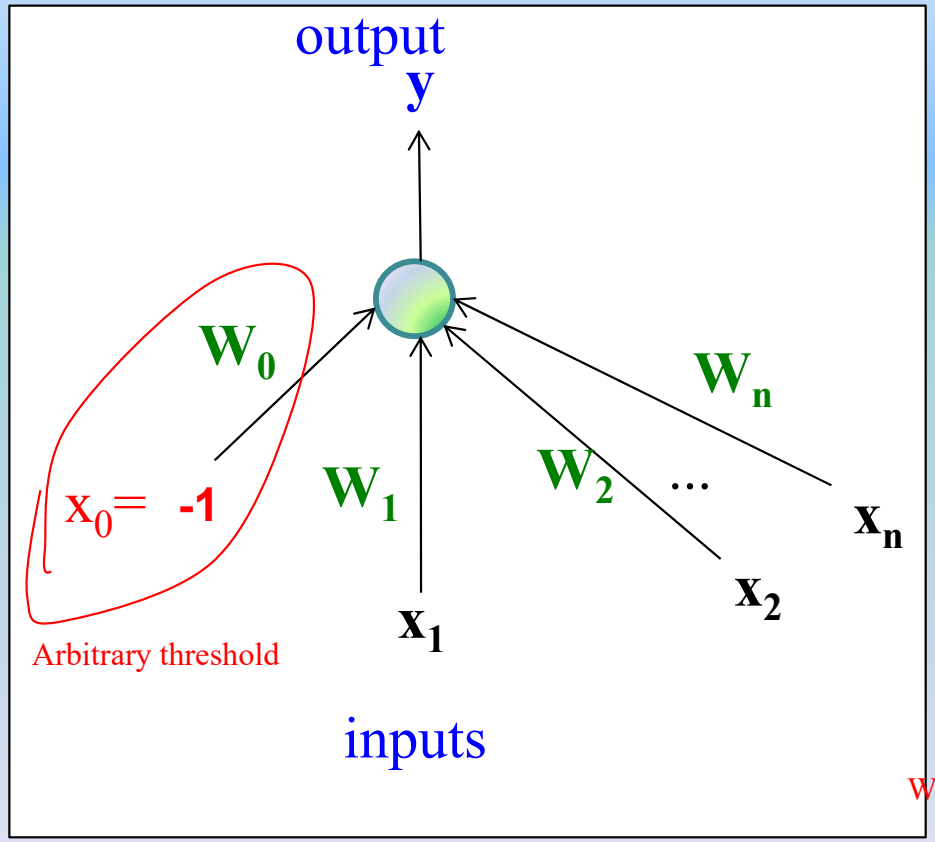
Single Perceptron Unit



A perceptron unit basically compares a weighted combination of its inputs against a **threshold** value and then outputs a **1** if the weighted inputs exceed the threshold.

$$h(x) = \theta(\mathbf{w} \cdot \mathbf{x} + b) \equiv \theta(\overline{\mathbf{w}} \cdot \overline{\mathbf{x}})$$

Single Perceptron Unit



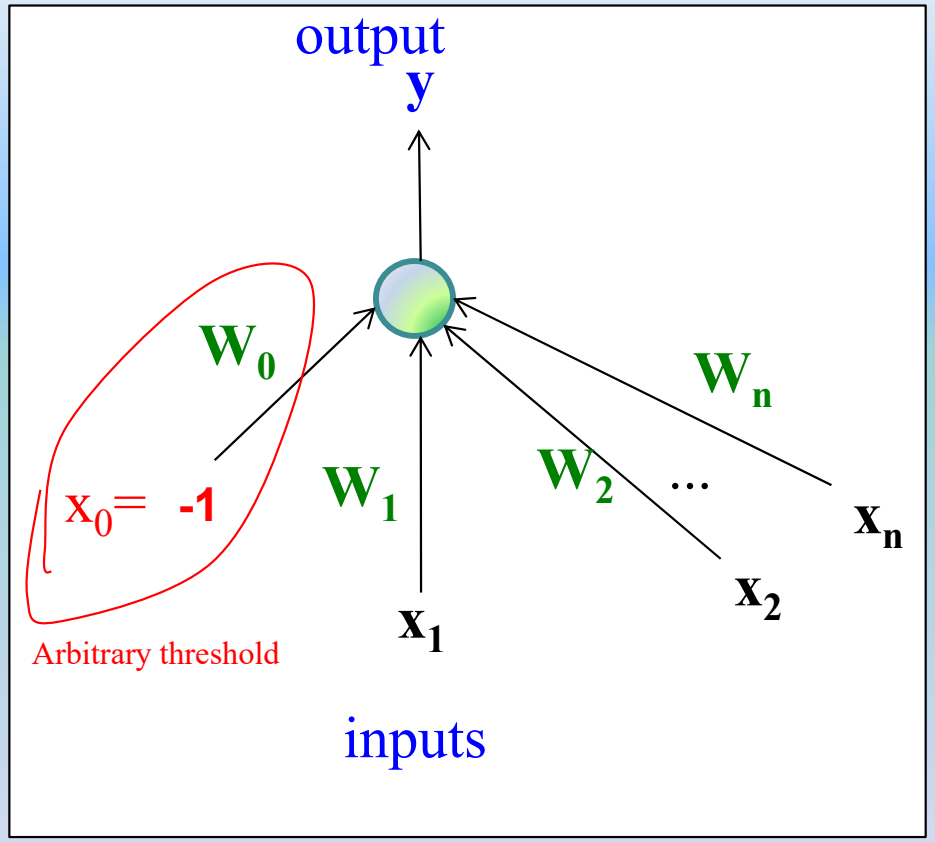
Basic rule of operation:

Compute the weighted sum of all the inputs then compare the result to zero.

$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Weighted input

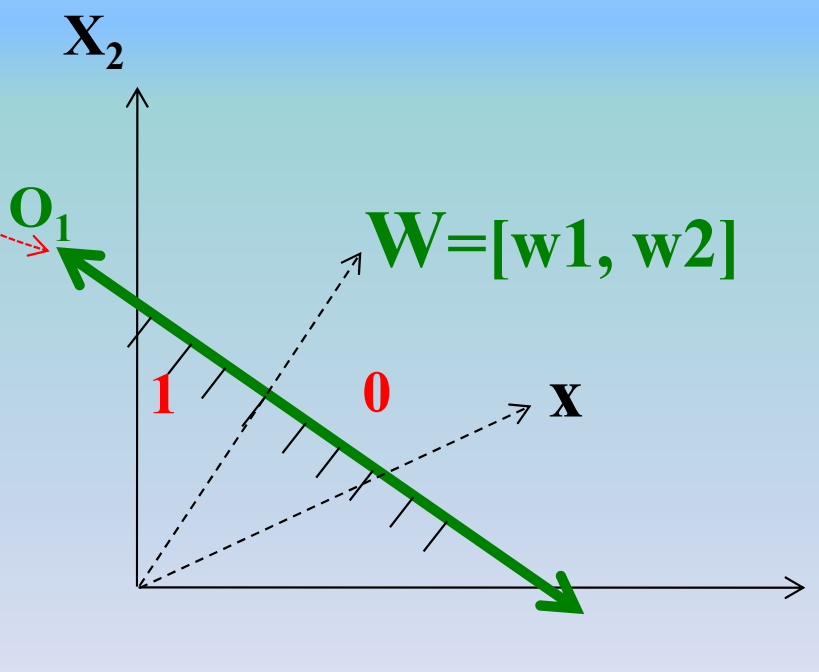
Linear Classifier Single Perceptron Unit



$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

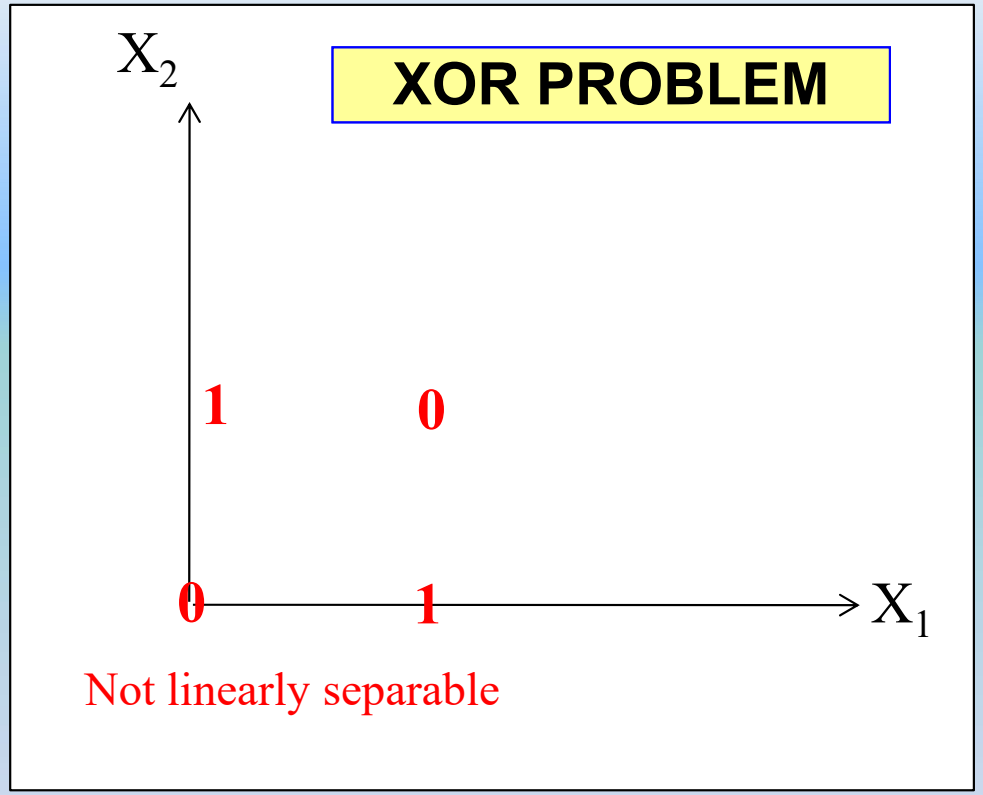
Weighted input

The decision boundary for a single perceptron is a hyperplane in the feature space.



Linear Classifier Single Perceptron Unit

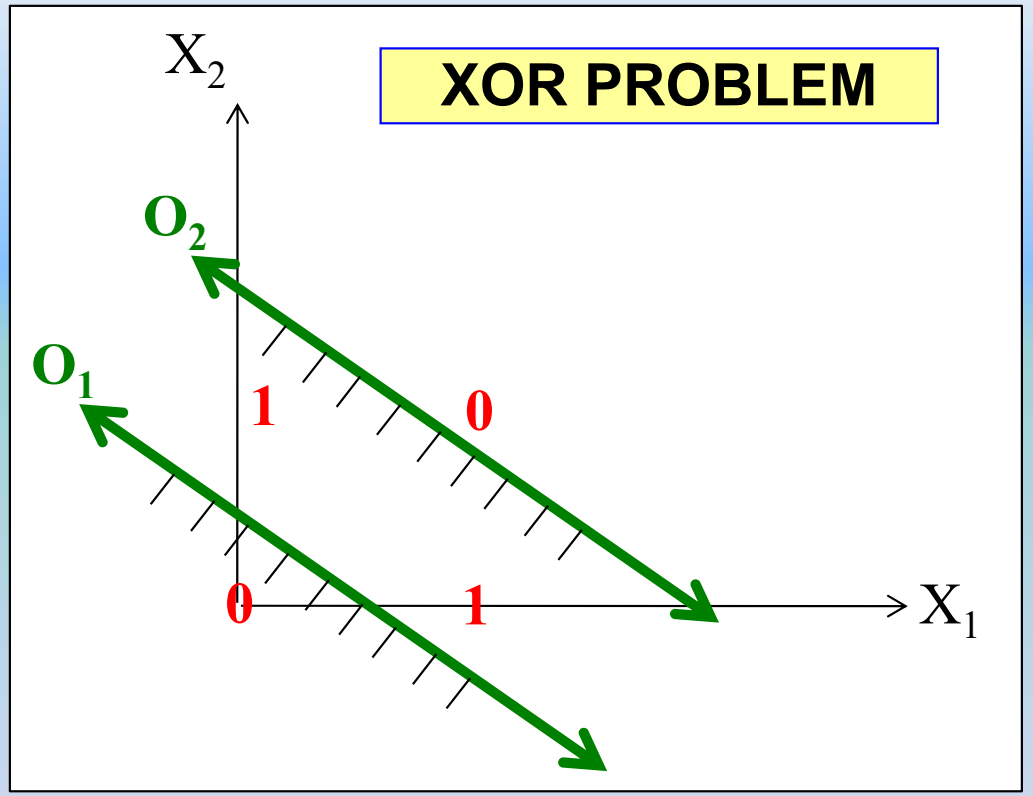
Beyond Linear Separability



Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

A problem like that illustrated by the values of the **XOR** boolean function **cannot** be solved by a **single perceptron unit**.

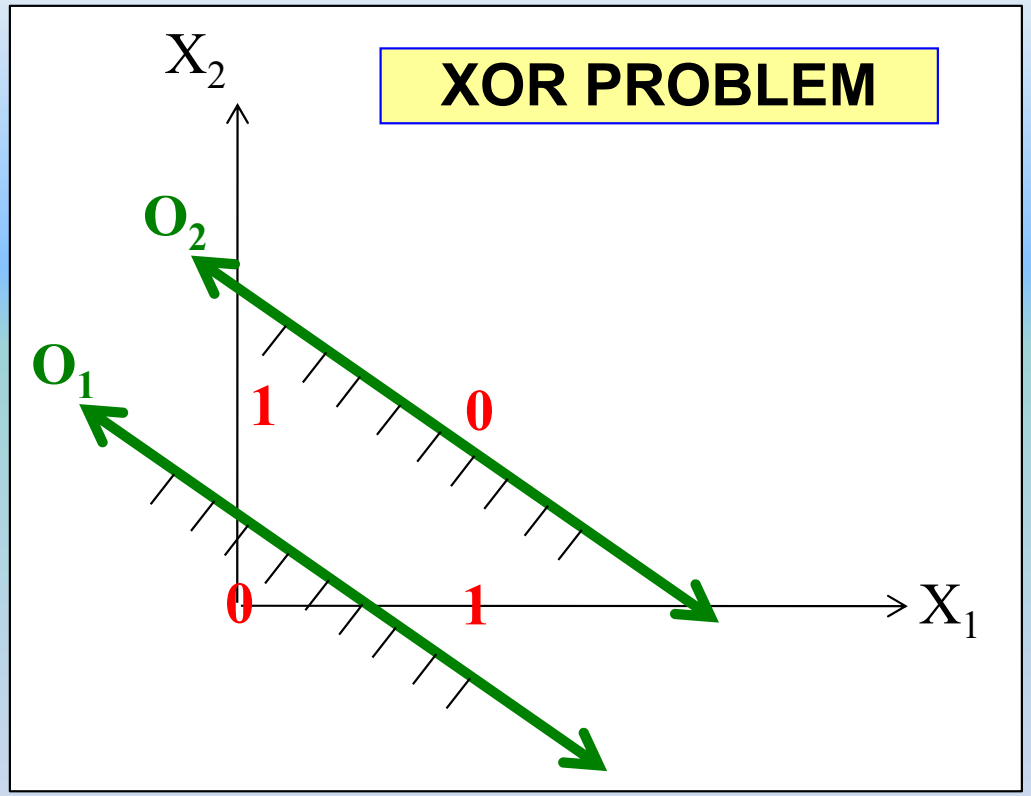
Beyond Linear Separability



Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Beyond Linear Separability

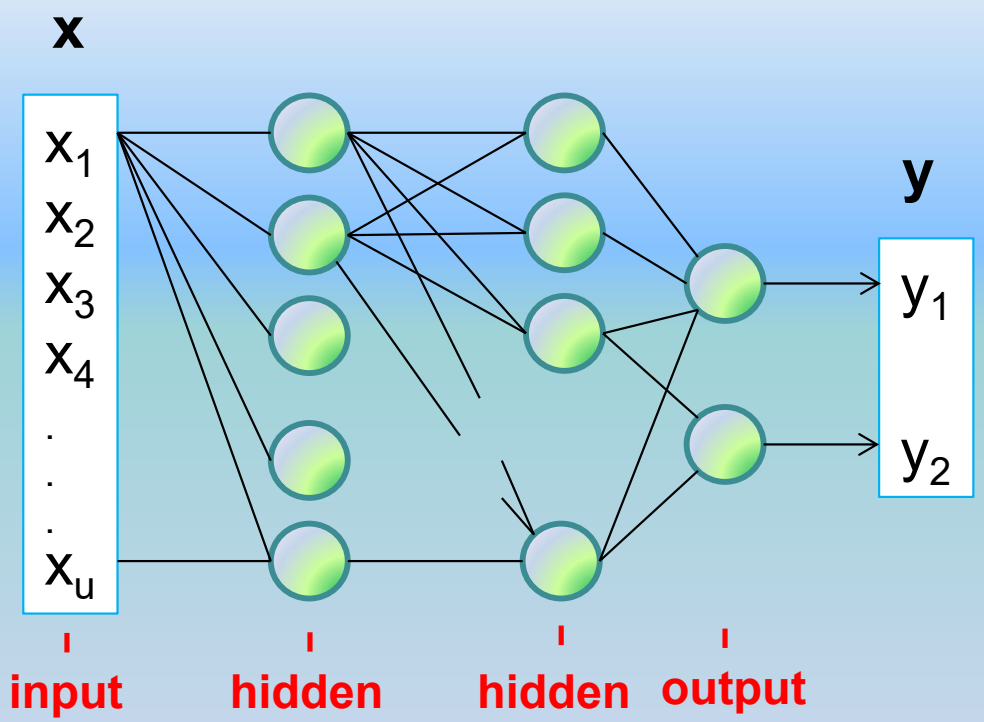


Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

Multi-Layer Perceptron



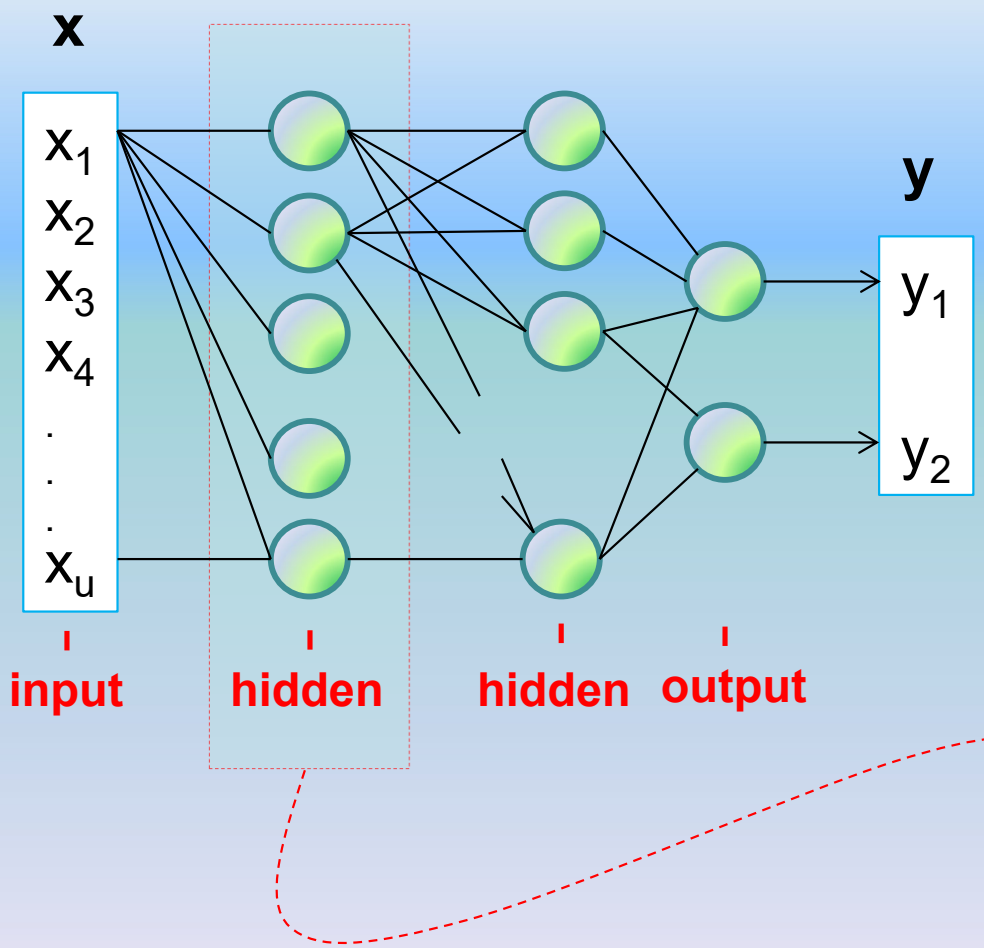
Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

The introduction of “hidden” (not connected directly to the output) units into these networks make them much more powerful: they are no longer limited to linearly separable problems.

Multi-Layer Perceptron



Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

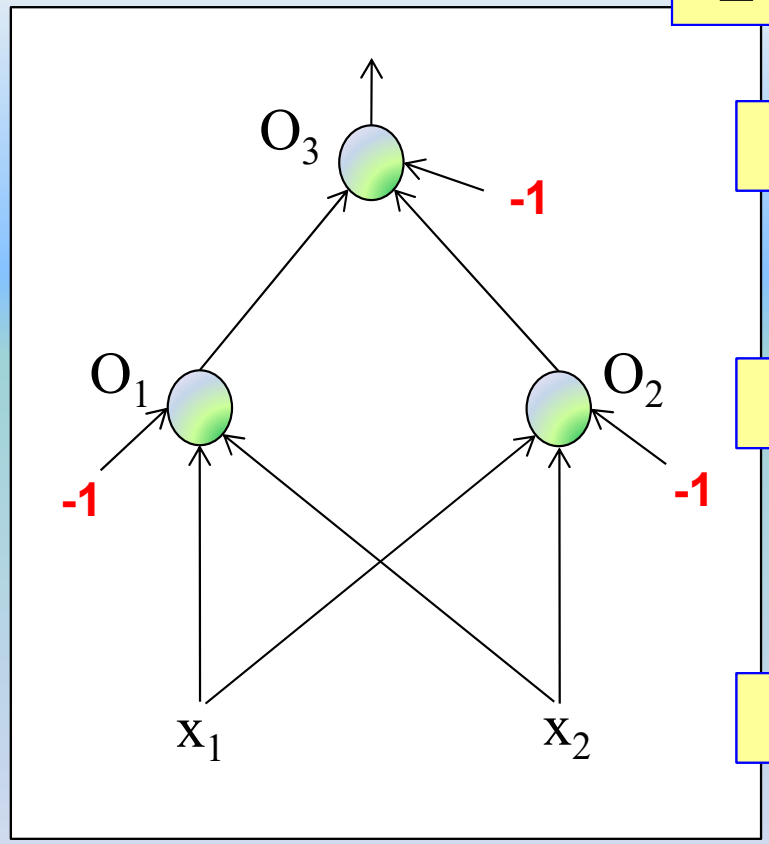
The introduction of “hidden” (not connected directly to the output) units into these networks make them much more powerful: they are no longer limited to linearly separable problems.

The lower layers transform the input problem into more tractable (linearly separable) problems for subsequent layers.

MLP for Solving the XOR Problem

Example

2-Layer Perceptron Network



Output units

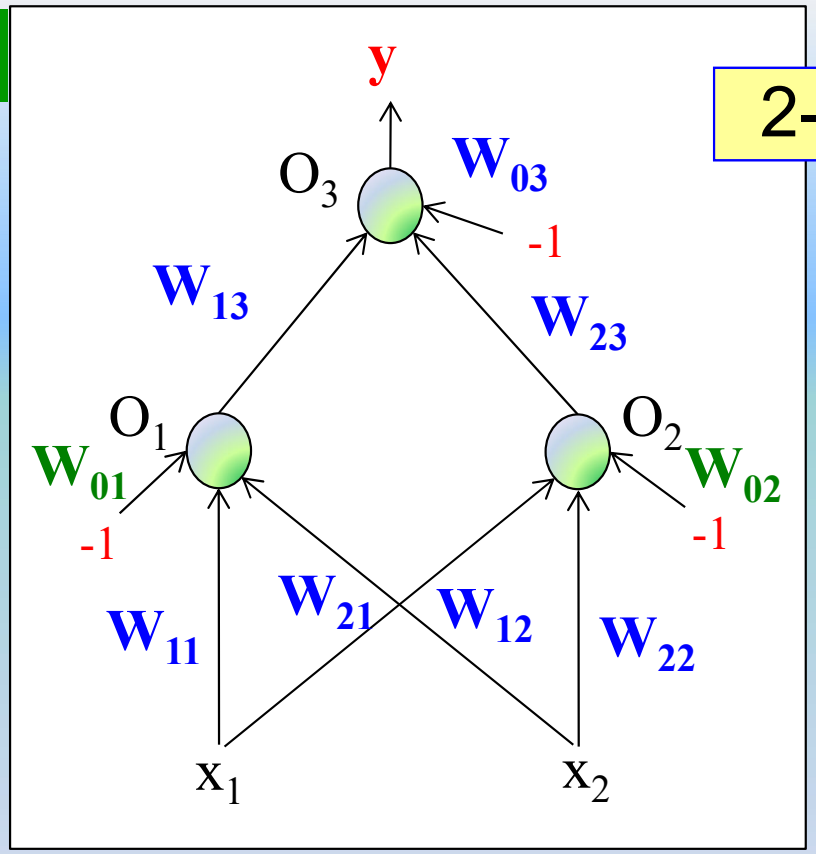
Hidden units

Input units

Interconnected network of simple units in which each connection has a weight.

Multilayer Layer Perceptron

Example



2-Layer Perceptron Network

Hidden units:

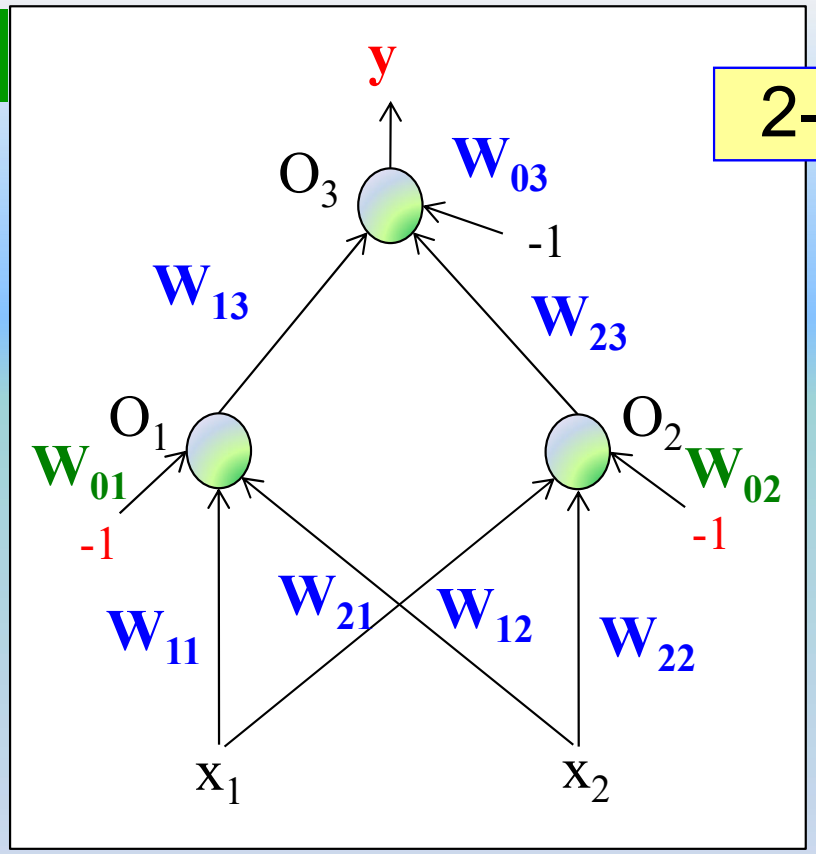
Weight for the link connecting input $\mathbf{x_1}$ to hidden node 2.

$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Multilayer Layer Perceptron

Example



2-Layer Perceptron Network

Hidden units:

$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

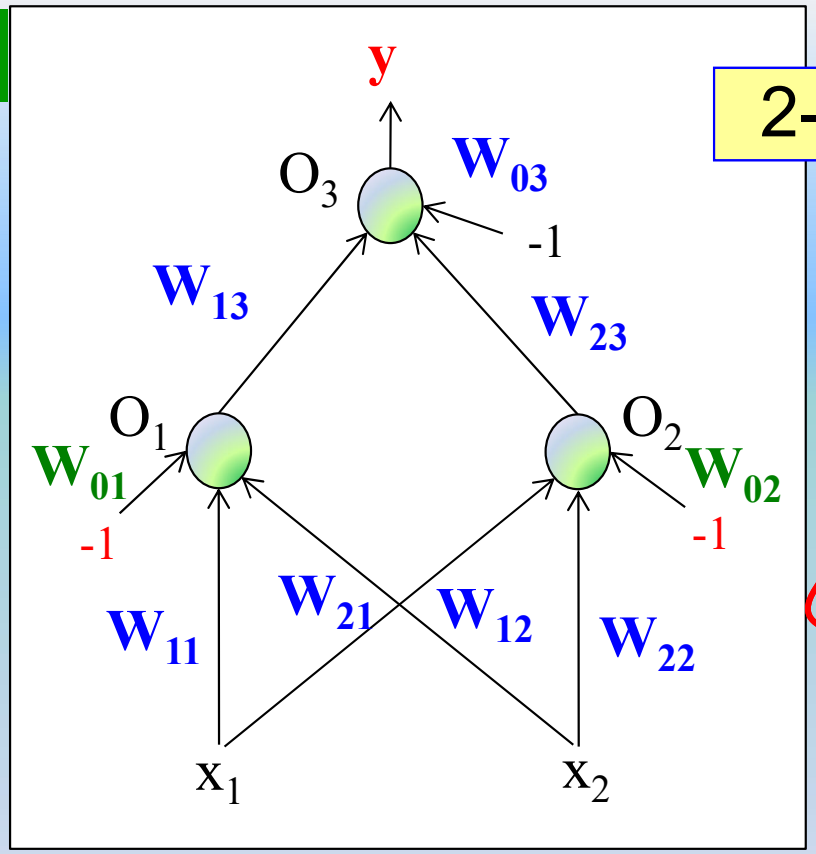
$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Each perceptron unit is a linear classifier, whose arbitrary threshold is defined by a weight W_{0i} and fixed input -1.

W_{0i} affects the **slope** and **offset** of the linear classifier from the origin (more on this later).

Multilayer Layer Perceptron

Example



2-Layer Perceptron Network

Hidden units:

$$(x_1 * w_{11}) + (x_2 * w_{21}) - w_{01} = O_1$$

$$(x_1 * w_{12}) + (x_2 * w_{22}) - w_{02} = O_2$$

Normal vector to the decision boundary

$$\mathbf{w}_1 = [w_{11}, w_{21}]$$

$$\mathbf{w}_2 = [w_{12}, w_{22}]$$

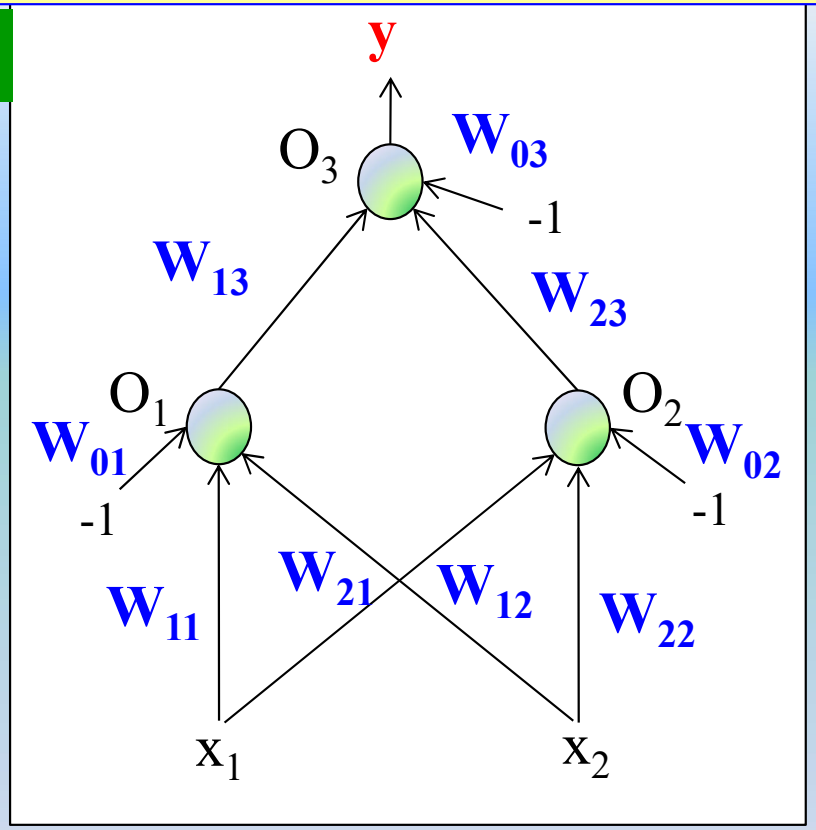
$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Multilayer Layer Perceptron

2-Layer Perceptron Network

Example



$$(x_1 * w_{11}) + (x_2 * w_{21}) - w_{01} = O_1$$

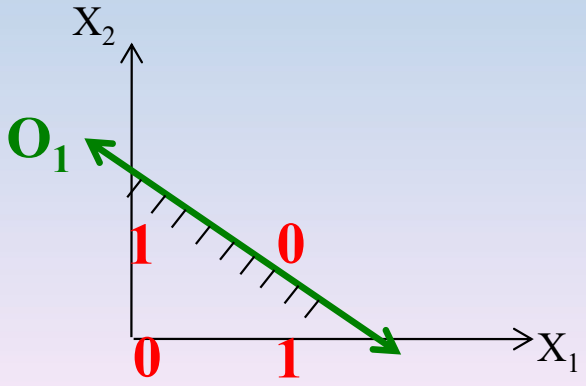
1 1 3/2

Slope of linear classifier $O_1 = -1$

Normal vector to the decision boundary

$$\mathbf{w}_1 = [w_{11}, w_{21}]$$

Offset (distance from origin) = 3/2

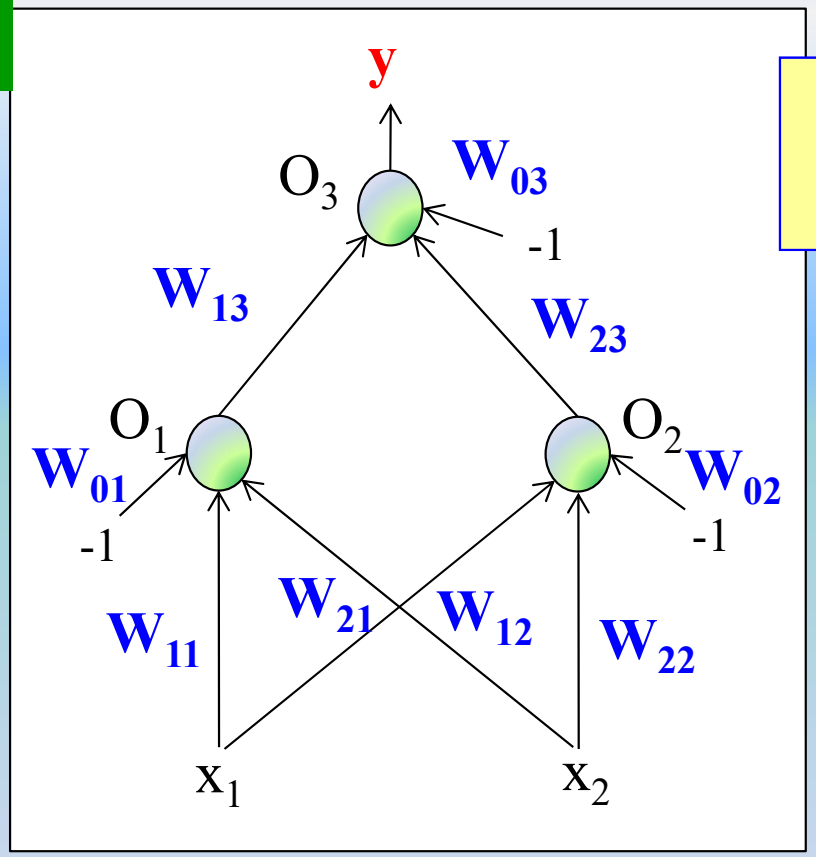


$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Multilayer Layer Perceptron

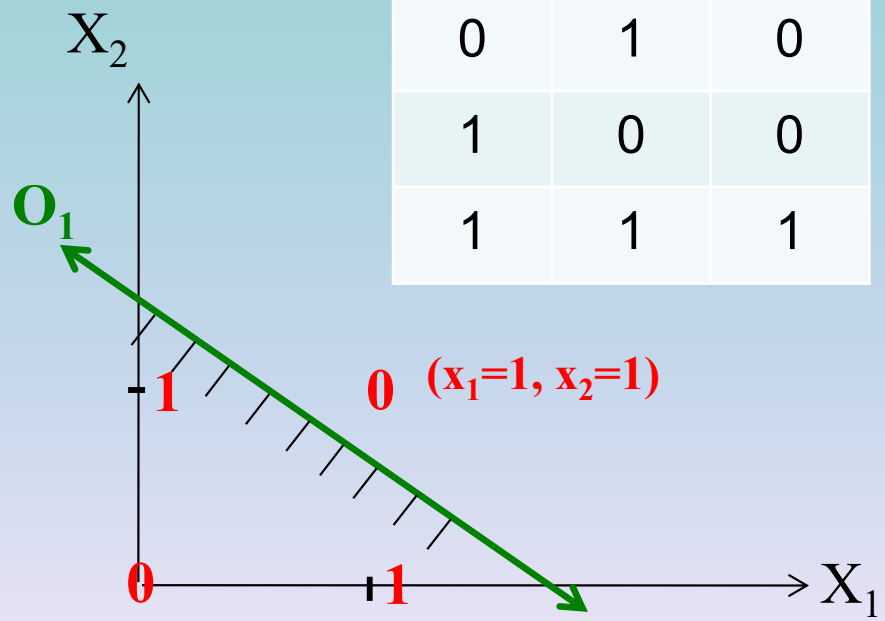
Example



HIDDEN UNIT (O_1)

DECISION BOUNDARY

X_1	X_2	O_1
0	0	0
0	1	0
1	0	0
1	1	1



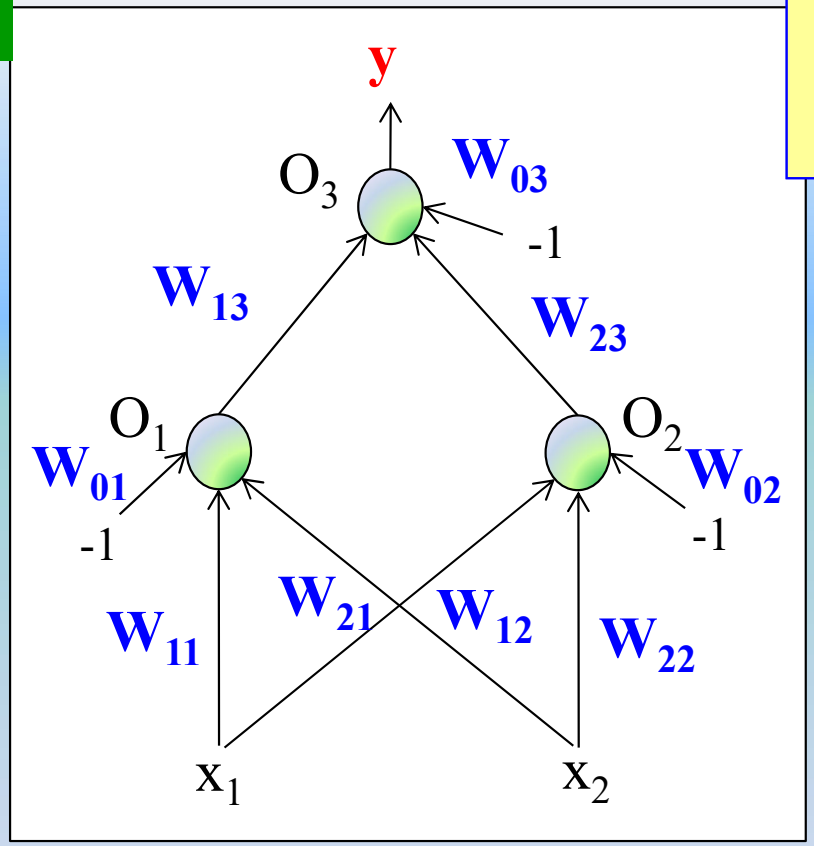
$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Each hidden unit defines its own “decision boundary” and the output from each of these units is fed to the output unit.

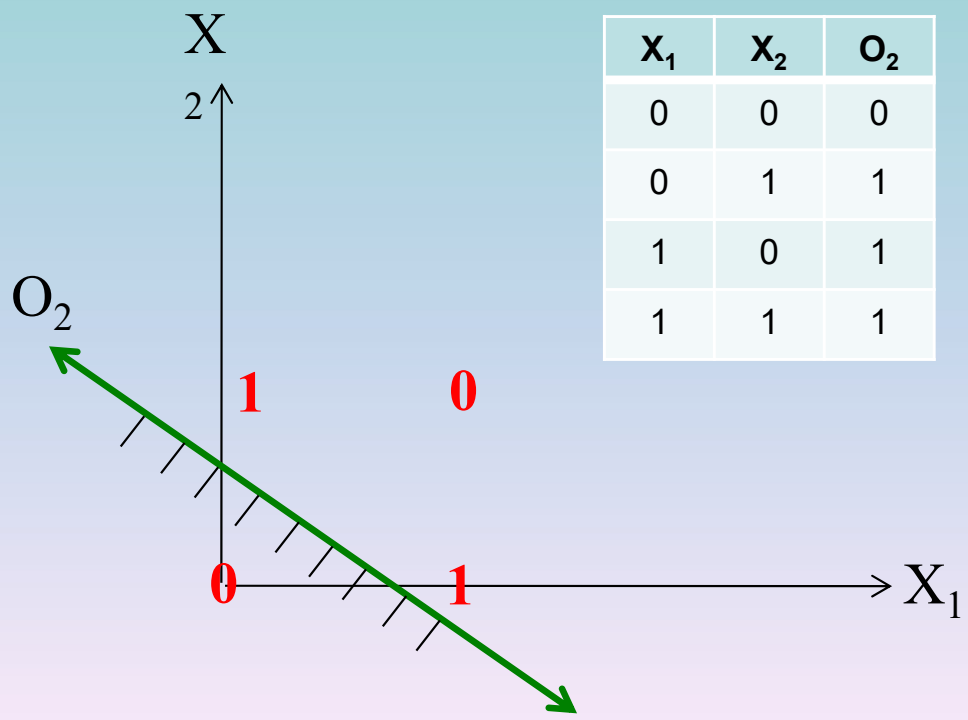
Multilayer Layer Perceptron

Example



HIDDEN UNIT (O_2)
DECISION BOUNDARY

$$(x_1 * w_{12}) + (x_2 * w_{22}) - w_{02} = O_2$$



X_1	X_2	O_2
0	0	0
0	1	1
1	0	1
1	1	1

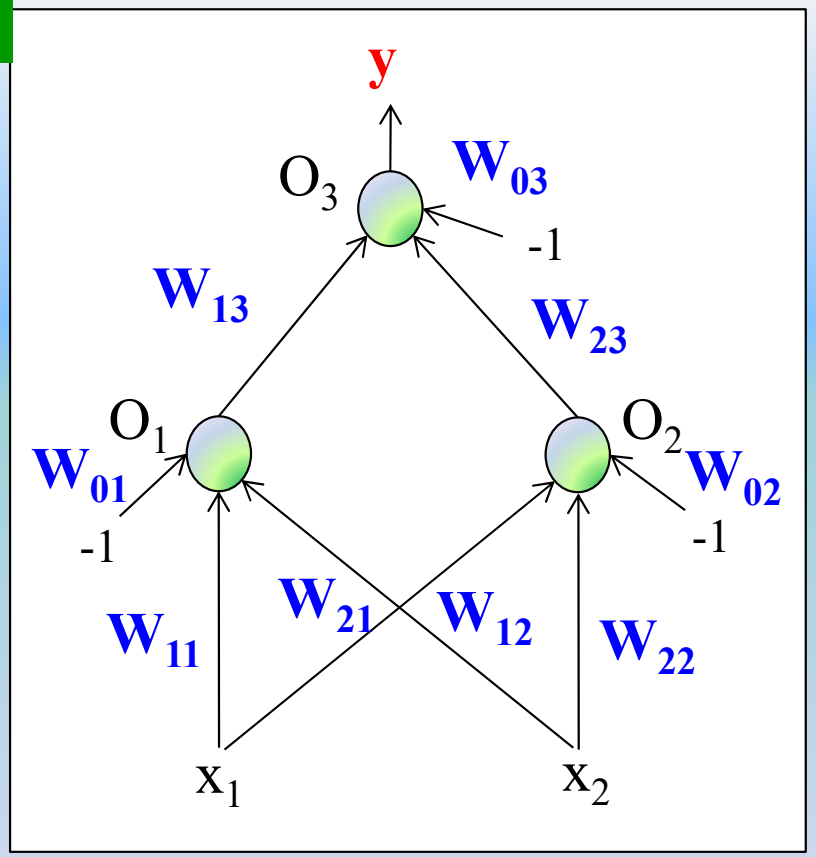
HIDDEN UNIT (O_2) DECISION BOUNDARY

$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

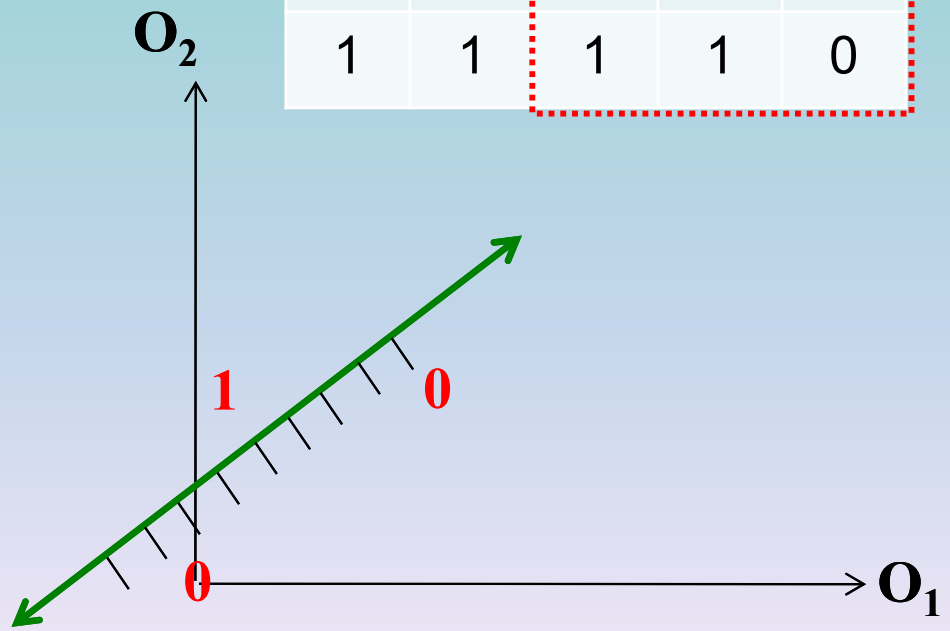
Multilayer Layer Perceptron

Example



XOR PROBLEM

X_1	X_2	O_1	O_2	Y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0



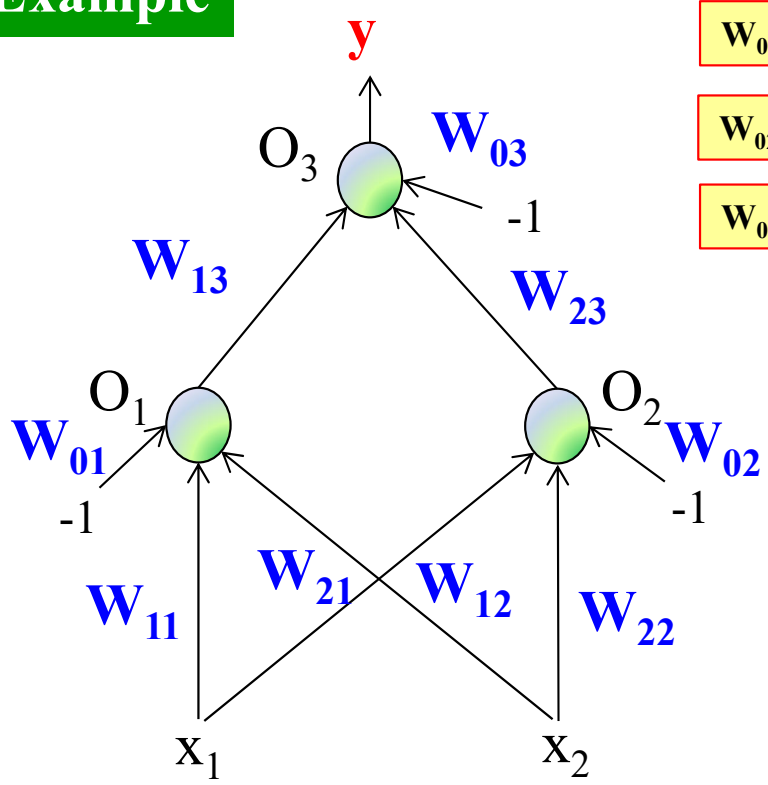
$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

$W_{03} = 1/2$ $W_{13} = -1$ $W_{23} = 1$

Multilayer Layer Perceptron

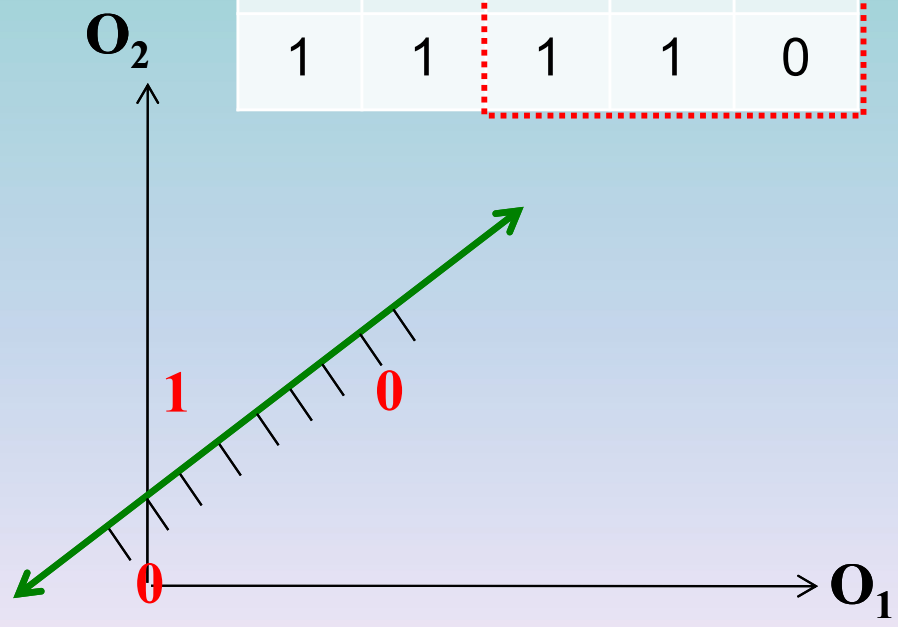
Example



$W_{01} = 3/2$	$W_{11} = 1$	$W_{12} = 1$
$W_{02} = 1/2$	$W_{21} = 1$	$W_{22} = 1$
$W_{03} = 1/2$	$W_{13} = -1$	$W_{23} = 1$

XOR PROBLEM

X_1	X_2	O_1	O_2	Y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0



The problem has been mapped into a linearly separable problem in the space of the outputs of the hidden units. We can now find a linear separator that solves the XOR problem.

Multilayer Layer Perceptron

Any set of training points can be separated by a three-layer perceptron network (with sufficiently many units).

“Almost any” set of points are separable by a two-layer perceptron network (with lots of units).

But no efficient learning rule is known.

Could we use gradient ascent/descent?

Threshold function is not smooth.
We need a smooth function in lieu of the threshold function.

We need a smooth output (as a function of changes in the network weights) in order for us to use gradient descent.